MAX's Graph Compiler

An E2E walkthrough of the optimizing pipeline



MAX is now **open source**, graph compiler included.

RMO and MO

Python → graph API → **RMO** → legalize → **MO** → mo-to-map → MAP → map-to-mogg → MOGG → mogg-to-mgp → MGP → emit-mojo → Mojo

One pipeline for every target: **all CPUs and GPUs** today, with **Qualcomm** and **Trainium** support in progress.

RMO Relaxed Modular Operations

Description	What the Python graph API emits. It accepts incomplete type information, and its ops behave like framework ops: a broadcasting add, a reduce that takes an axis.
Example ops	**rmo.mul**, **rmo.add**, **rmo.mo.reduce.add**, ...
Optimizations	None: **mo-legalize-rmo-operators** rewrites it into **MO**.

```
%2 = rmo.mo.reduce.add(%1) [axis = 1]
```

MO Modular Operations

Description	Framework-agnostic, strongly typed graph ops with value semantics. Each op does one thing (**mo.add** does not broadcast) and shapes must provably match.
Example ops	**mo.mul**, **mo.add**, **mo.reduce.add**, ...
Optimizations	Canonicalization, constant folding, symbolic shape simplification.

```
%3 = mo.reduce.add(%2)
```



MAP and MOGG

Python → graph API → RMO → legalize → MO → mo-to-map → **MAP** → map-to-mogg → **MOGG** → mogg-to-mgp → MGP → emit-mojo → Mojo

MAP Modular Access Primitives

Description	Kernels as loop nests over index spaces that load, store and compute. Hand-written kernels (**map.iter.opaque**) fuse through their prologue and epilogue.
Ops	**map.iter.elementwise**, **map.iter.rowwise**, **map.iter.opaque**, **map.call.elementwise**
Optimizations	Fusion (**map-fuse-kernels**): elementwise chains, views, prologues and epilogues, rowwise reductions, until nothing fuses.

MOGG Modular Operation Generator Graph

Description	Kernel selection and codegen, produced from MAP after fusion: which Mojo kernel runs, on pointers, shapes and SIMD values.
Ops	**mogg.routine**, **mogg.call.rowwise**, **mogg.func**, **mogg.with_trace**
Optimizations	Rowwise launch plans, outlined kernel stubs (**mogg-outline-routines**), conformed call operands.



MGP and Mojo

Python → graph API → RMO → legalize → MO → mo-to-map → MAP → map-to-mogg → MOGG → mogg-to-mgp → **MGP** → emit-mojo → **Mojo**

MGP Modular Glue Primitives

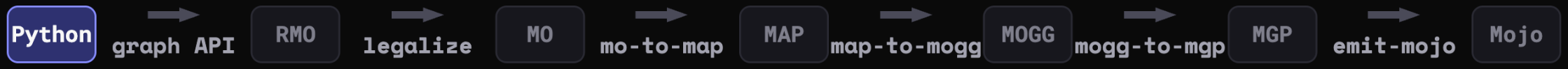
Description	Runtime glue with buffer semantics: buffers, device contexts and async values bind kernels into one model, translated directly to MEF.
Ops	**mgp.model**, **mgp.buffer.alloc**, **mgp.tensor.create**, async entry stubs
Optimizations	Memory planning and buffer reuse, exec-invariant code motion, merging identical functions; the debug passes run here too.

Mojo The kernels, as source

Description	Every kernel stub and entry stub, printed by **mogg-emit-mojo** and compiled by the Mojo compiler (**jit**) into the MEF.
Ops	Functions and closures over the kernel library
Optimizations	The Mojo compiler's: specialization on parameters, inlining; compiled kernels are cached.



Python


Python
graph API
RMO
legalize
MO
mo-to-map
MAP
map-to-mogg
MOGG
mogg-to-mgp
MGP
emit-mojo
Mojo

Model Building in Python

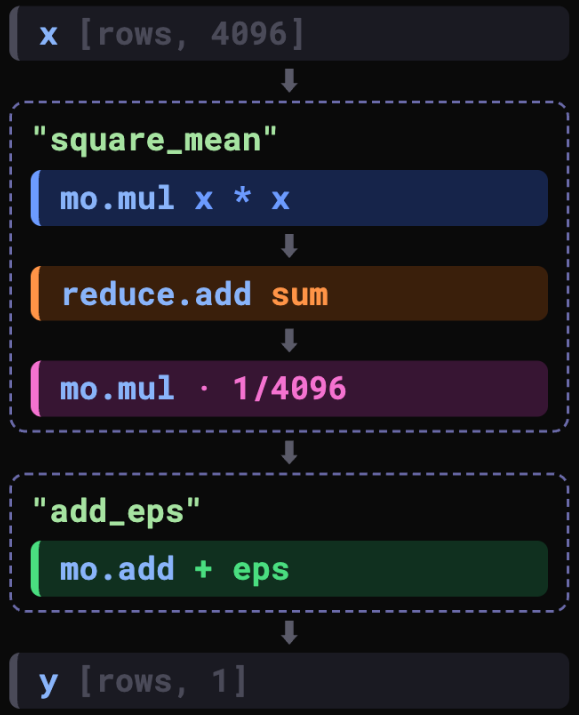
`y = F.mean(x * x, axis=-1) + EPS`

- Each line becomes graph ops inside its profile scope.
- **rows** stays symbolic until run time.
- Each op also keeps the Python frame that created it, as an opaque **loc**.

rms_demo.py

```
class SquareMeanEps(Module[[Tensor], Tensor]):
    def forward(self, x: Tensor) -> Tensor:
        with Graph.current.profile_scope("square_mean"):
            ms = F.mean(x * x, axis=-1)
        with Graph.current.profile_scope("add_eps"):
            return ms + EPS

def main() -> int:
    rows, cols = 8, 4096
    model = SquareMeanEps()
    run = model.compile(TensorType(DType.float32, ["rows", cols], ...))
    y = run(Tensor.from_dlpack(x_np).to(device))
```



Keeping Track of Provenance with a Stack Trace

- The graph API attaches an **OpaqueLoc** to each op: a live pointer to the Python frame that created it.
- The IR prints it as **loc(unknown)**; **mgp-attach-python-stack-traces** turns it into text.

rms_demo.py

```
48  class SquareMeanEps(Module[[Tensor], Tensor]):
49      def forward(self, x: Tensor) -> Tensor:
51              ms = F.mean(x * x, axis=-1)
...
60      run = model.compile(TensorType(...))
...
71      sys.exit(main())
```

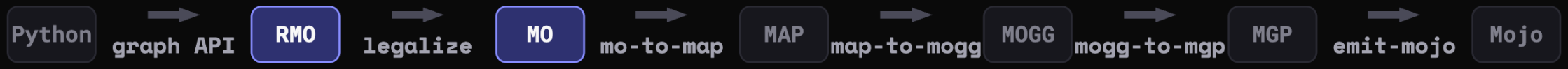
The stack its opaque **loc** holds

```
File ".../rms_demo.py", line 71, in <module>
  sys.exit(main())
File ".../rms_demo.py", line 60, in main
  run = model.compile(TensorType(...))
... frames inside MAX ...
File ".../rms_demo.py", line 51, in forward
  ms = F.mean(x * x, axis=-1)
```

The **x * x** op as the graph API builds it

```
%1 = rmo.mul(%arg0, %arg0) : (...) -> !mo.tensor<[rows, 4096], f32, gpu:0> loc(#loc9330)
#loc9330 = #mogg.profile_scope<"square_mean", #loc4758>
#loc4758 = loc(#loc) // the OpaqueLoc: it prints as unknown, but holds the Python frame
```

RMO and MO


Python
graph API
RMO
legalize
MO
mo-to-map
MAP
map-to-mogg
MOGG
mogg-to-mgp
MGP
emit-mojo
Mojo

RMO Legalization

- **mo-legalize-rmo-operators** rewrites each relaxed **RMO** op into strict **MO** ops.
- Broadcasts become explicit, and every shape is checked.

`mean(x * x) + eps = sum(x * x) · 1/4096 + eps`

RMO, as the Python graph API builds it

```
mo.graph @SquareMeanEps<rows>(%arg0: !x) ... {
  %1 = rmo.mul(%arg0, %arg0) : (!x, !x) -> !x
  %2 = rmo.mo.reduce.add(%1) [axis = 1] ...
  %3 = mo.constant <... 2.44140625E-4 ...> : !s
  %4 = rmo.mul(%2, %3) : (!y, !s) -> !y
  %5 = mo.constant <... 9.99999997E-7 ...> : !s
  %6 = rmo.add(%4, %5) : (!y, !s) -> !y
  mo.output %6 : !y
}
```

MO, after legalization

```
mo.graph @SquareMeanEps<rows>(%arg0: !x) ... {
  %0 = mo.constant <... 9.99999997E-7 ...> : !s
  %1 = mo.constant <... 2.44140625E-4 ...> : !s
  %2 = mo.mul(%arg0, %arg0) : !x
  %3 = mo.reduce.add(%2) : (!x) -> !y
  %4 = mo.static.broadcast_to(%1) : !s -> !y
  %5 = mo.mul(%3, %4) : !y
  %6 = mo.static.broadcast_to(%0) : !s -> !y
  %7 = mo.add(%5, %6) : !y
  mo.output %7 : !y
}
```

`!x = !mo.tensor<[rows, 4096], f32, gpu:0> !y = !mo.tensor<[rows, 1], f32, gpu:0> !s = !mo.tensor<[], f32, gpu:0>`

Shapes can be **static, dynamic or hybrid**: here **rows** is dynamic and **4096** is static.

Kernel Registration

- **mogg-import-packages** writes a **mogg.kernel.decl** per registered struct.
- **x * x** runs **Mul**, **the sum** runs **ReduceSum**, **+ eps** runs **Add**.

builtin_kernels/elementwise.mojo, **monoids.mojo**

```
@extensibility.register("mo.mul")
struct Mul(ElementwiseBinaryOp):
    @staticmethod
    def elementwise[
        dtype: DType,
        width: SIMDLength,
    ](lhs: SIMD[dtype, width], rhs: ...) -> ...:
        return lhs * rhs
@extensibility.register("mo.reduce.add")
struct ReduceSum[
    dtype: DType,
    W: Int = simd_width_of[dtype](),
](ReduceOp):
    var acc: SIMD[Self.dtype, Self.W]
```

The decls **mo-to-map** reads

```
mogg.kernel.decl
  @builtin_kernels.elementwise.Mul
  registration = "mo.mul" ...
  elementwise = <<!lit.generator<
    <"dtype": !DType, ...>
    ("lhs": ..., "rhs": ...) -> ...>>, ...


mogg.kernel.decl
  @builtin_kernels.monoids.ReduceSum
  registration = "mo.reduce.add" ...
  reduction = <<"dtype": !DType, ...>,
    "W", [...]>
```

@extensibility.register("mo.add", type="gpu", api="cuda") registers a kernel per device: all CPUs and GPUs, soon Qualcomm and Trainium.

Kernel Verification

- **mogg-verify-kernels** checks every kernel's signature while the graph is built.
- It runs when a custom-op package loads, and on every **ops.custom** call.

kernels/my_op.mojo

```
@extensibility.register("my_op")
struct MyOp:
    @staticmethod
    def execute[
        target: StaticString
    ](
        x: InputTensor,
        output: OutputTensor[dtype=x.dtype, ...],
        ctx: DeviceContext,
    ) raises:
        pass
```

The error, at graph build

```
max.graph.KernelRegistrationError:
.../kernels/my_op.mojo:12:9: error: Outputs
must appear before inputs in the function
signature: output 'output' comes after input 'x'
        output: OutputTensor[dtype=x.dtype, ...],
        ^
```

MAP and MOGG


Python
graph API
RMO
legalize
MO
mo-to-map
MAP
map-to-mogg
MOGG
mogg-to-mgp
MGP
emit-mojo
Mojo

Lowering MO to MAP

- **mo-to-map** lowers each **MO** op to a **MAP** kernel that calls its registered decl: **@Mul** is **@builtin_kernels.elementwise.Mul**.
- Broadcasts become views.

`mean(x * x) + eps = sum(x * x) · 1/4096 + eps`

MO

```
mo.graph @SquareMeanEps<rows>(%arg0: !x) ... {
  ...
  %2 = mo.mul(%arg0, %arg0) : !x
  %3 = mo.reduce.add(%2) : (!x) -> !y
  %4 = mo.static.broadcast_to(%1) : !s -> !y
  %5 = mo.mul(%3, %4) : !y
  %6 = mo.static.broadcast_to(%0) : !s -> !y
  %7 = mo.add(%5, %6) : !y
  mo.output %7 : !y
}
```

MAP, after **mo-to-map**

```
%2 = map.iter.elementwise<[rows, 4096]>
  index (%arg4: !idx) -> !x {
    %10 = map.call.elementwise @Mul(...)
  } {fusionInfo = #mul, ...}
%3 = map.iter.rowwise<[rows, 4096], axis = 1>
  row (%arg3: !idx) -> !y {
    %acc = map.iter.reduce @ReduceSum ...
    map.store %arg2[%arg3] = %acc : !y
  } {fusionInfo = #reduce_add}
... the broadcast views and the * 1/N kernel ...
%7 = map.iter.elementwise<[rows, 1]>
    %10 = map.call.elementwise @Add(...)
  } {fusionInfo = #add, ...}
```

MAP Looks Like MLIR's affine Dialect

MAP (compiler output)

```
%2 = map.iter.elementwise<[rows, 4096]>
  index (%arg4: !idx) -> !x {
    %8 = map.load %arg1[%arg4] : !x
    %10 = map.call.elementwise @Mul(...)
    map.store %arg3[%arg4] = %10 : !x
  }
%3 = map.iter.rowwise<[rows, 4096], axis = 1>
  row (%arg3: !idx) -> !y {
    %acc = map.iter.reduce @ReduceSum ...
      %8 = map.load %2[%arg4] : !x
    map.store %arg2[%arg3] = %acc : !y
```

The same loops in **affine** (hand-written, for comparison)

```
affine.parallel (%i, %j) = (0, 0) to (symbol(%n), 4096) {
  %a = affine.load %x[%i, %j] : memref<?x4096xf32>
  %p = arith.mulf %a, %a : f32
  affine.store %p, %t[%i, %j] : memref<?x4096xf32>
}
affine.parallel (%i) = (0) to (symbol(%n)) {
  %s = affine.parallel (%j) = (0) to (4096) reduce ("addf")
    -> f32 {
    %v = affine.load %t[%i, %j] : memref<?x4096xf32>
    affine.yield %v : f32
  affine.store %s, %out[%i, 0] : memref<?x1xf32>
```

map.iter.elementwise, map.index<2>	**affine.parallel (%i, %j)**
map.load / map.store	**affine.load / affine.store**
map.iter.reduce @ReduceSum	**affine.parallel ... reduce ("addf")**
map.index.broadcast	**an affine map (d0, d1) -> ()**

Structured Kernel Fusion

- **map-fuse-kernels** inlines producers into consumers (epilogue, elementwise, view, prologue) until a round fuses nothing.
- **x * x** moves into **the reduce**; **· 1/4096** and **+ eps** run once per row, in its epilogue.

The fused kernel, in full

```
%2 = map.iter.rowwise<[rows, 4096], axis = 1>
inputs (%arg0: !x, %1: !s, %0: !s)
outputs (%arg4: !y)
row (%arg5: !idx) -> !y {
  %acc = map.iter.reduce @ReduceSum index (%arg6: !idx) -> f32 {
    %8 = map.load %arg0[%arg6] : !x
    %9 = map.call.elementwise @Mul(%arg6, %8, %8) : !idx, f32, f32 -> f32
    map.yield %9 : f32
  }
  %3 = map.index.broadcast<#map.index.broadcast.spec<[] -> [rows, 1]>> (%arg5) : !idx -> !map.index<0>
  %4 = map.load %1[%3] : !s
  %5 = map.call.elementwise @Mul(%arg5, %acc, %4) : !idx, f32, f32 -> f32
  %6 = map.load %0[%3] : !s
  %7 = map.call.elementwise @Add(%arg5, %5, %6) : !idx, f32, f32 -> f32
  map.store %arg4[%arg5] = %7 : !y
} {fusionInfo = #mul_reduce_add_and4, hasReadEffect = false, hasWriteEffect = false}
```

Fusion Info

```
#mul = #mogg.fusion<#mogg.fusion_leaf<"mo.mul", loc = ..., sig = (!x, !x) -> !x>>
#reduce_add = #mogg.fusion<#mogg.fusion_leaf<"mo.reduce.add", loc = ..., sig = (!x) -> !y>>
#add = #mogg.fusion<#mogg.fusion_leaf<"mo.add", loc = ..., sig = (!y, !y) -> !y>>
// map-fuse-kernels merges the leaves, and their locations, into one tree:
#mul_reduce_add_and4 = #mogg.fusion<#mogg.fusion_rowwise<[
  #mogg.fusion_leaf<"mo.mul", ...>,
  #mogg.fusion_leaf<"mo.reduce.add", ...>,
  #mogg.fusion_view<[
    #mogg.fusion_leaf<"mo.static.broadcast_to", ...>,
    #mogg.fusion_leaf<"mo.static.broadcast_to", ...>,
    #mogg.fusion_elementwise<[#mogg.fusion_leaf<"mo.mul", ...>, #mogg.fusion_leaf<"mo.add", ...>]>]>]>>
```

Kernel name, from **getFuncName**

__mul_reduce_add_2ba342a67ef6a164

Trace name, from **toString**

Rowwise(mul, reduce.add, View(...)) [add_eps:square_mean]

Comment and error tree, from **toPrettyTree**

```
# FusionInfo:
# Rowwise
# ├── mo.mul : ...
```

MOGG: Progressive Lowering to Mojo

- **MOGG** bridges the fusion dialect and the emitted Mojo: each op records what the Mojo printer needs.
- Tensors become **!mogg.pointer**s, plus **!mogg.dim**s for symbolic dims and a **!mogg.context**; loads give SIMD values.
- Every call names the Mojo symbol it runs: **@builtin_kernels.elementwise.Mul**, **@builtin_kernels.monoids.ReduceSum**.

After **map-to-mogg**

```
%0 = mogg.routine (%arg0 : !x) -> !y
  inputs (%arg1) // !mogg.pointer
  outputs (%arg2) // !mogg.pointer
  context (%arg4) // !mogg.context
  dims (%arg3: !mogg.dim<rows>) attributes {...} {
  %1 = mogg.shape.create(%arg3) <shape = #mosh<ape[dyn, 4096]> : !mosh.ape> : !mogg.dim<rows>
  %2 = mogg.tensor.create(%arg1, %1) <device = "gpu", elementType = f32, io = Input, ...> : !mogg.tensor
  ... the same for the output, %4 ...
  mogg.call.rowwise shape(%5) axis(1) context(%arg4) (%row : !idx) capturing {%cap0 = %2, %cap1 = %4} {
    %8 = mogg.rowwise.reduce @builtin_kernels.monoids.ReduceSum (%i : !idx) -> !mogg.simd<f32> {
      %11 = mogg.tensor.load %cap0[%i] : !mogg.tensor, !idx -> !mogg.simd<f32>
      %12 = mogg.call.elementwise @builtin_kernels.elementwise.Mul(%11, %11) moOpName = "mo.mul" ...
    }
    ... the epilogue: Mul, Add, then a store of the row ...
  }
```

Rowwise Legalization

- A rowwise kernel runs through a Mojo primitive, **_rowwise**, that needs compile-time parameters.
- **mogg-legalize-rowwise-kernels** derives them from the IR and records them on **mogg.call.rowwise**.

The rowwise primitive, **builtin_primitives**

```
def _rowwise[
    Body: RowBody, //,
    SimdMonoid: ReduceOp,
    dtype: DType,
    accum: DType,
    *,
    axis: Int,
    target: StaticString,
    num_phases: Int,
    computationally_expensive: Bool,
    dtype_size: Int,
](body: Body, shape: Coord,
    ctx: DeviceContext) raises:
```

After **mogg-legalize-rowwise-kernels**

```
mogg.call.rowwise shape(%0) axis(1) ... attributes {
    accum = f32,
    cols = 4096 : i64,
    dtype = f32,
    dtype_size = 0 : i64,
    num_phases = 1 : i64,
    primary = 0 : i64,
    simd_monoid = @builtin_kernels.monoids.ReduceSum
} {
    %6 = mogg.rowwise.reduce @ReduceSum ... {
      %9 = mogg.rowwise.primary : !mogg.simd<f32>
    } {fields = ["acc"], params = {dtype = f32}}
```

Function Outlining and Deduplication

- **mogg-outline-routines** moves each kernel body into its own **mogg.func**, named after its fusion info.
- **mogg-merge-identical-funcs** then merges identical functions, so each is emitted and compiled once.
- LLMs repeat the same layers many times, so most of their kernels are duplicates: this cuts compile time.

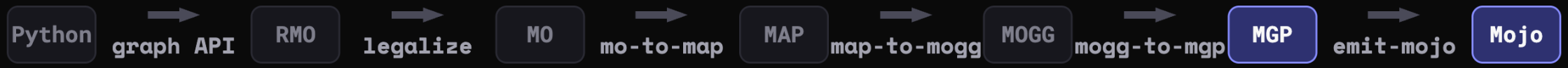
Before: the kernel body inside the routine

```
%0 = mogg.routine (%arg0 : !x) -> !y
  inputs (%arg1) outputs (%arg2)
  context (%arg4) dims (%arg3: ...) {
  %1 = mogg.shape.create(%arg3) ...
  %2 = mogg.tensor.create(%arg1, %1) ...
  ...
  mogg.call.rowwise shape(%5) ... {
    ...
  }
}
```

After **mogg-outline-routines**: a function and a call

```
%0 = mogg.routine (%arg0 : !x) -> !y ... {
  mogg.call @__mul_reduce_add_2ba342a67ef6a164
    %arg1, %arg2, %arg3, %arg4 : !mogg.pointer,
    !mogg.pointer, !mogg.dim<rows>, !mogg.context
}
mogg.func @__mul_reduce_add_2ba342a67ef6a164
  (%arg0: !mogg.pointer, %arg1: !mogg.pointer,
  %arg2: !mogg.dim<rows>, %arg3: !mogg.context) {
  ... the body, unchanged ...
}
```

MGP and Mojo


Python
graph API
RMO
legalize
MO
mo-to-map
MAP
map-to-mogg
MOGG
mogg-to-mgp
MGP
emit-mojo
Mojo

Async Routines

- **mogg-to-mgp** turns each **kernel call** into a **mogg.routine.async**: it **unpacks** its async inputs and calls the kernel stub.
- **mgp-lower-primitives** does the same for every **MGP primitive**: unpack, run, **pack**.

After **mogg-to-mgp**: the kernel is an async routine

```
execute (..., %arg5: !mgp.tensor<?x4096xf32, ...>) {
  %0 = mgp.tensor.extract.tensor_spec %arg5
  ... get_dim, casts, the output size %8 ...
  %9 = mgp.buffer.alloc %arg0, %8, %arg4, align 256
  %10 = mogg.routine.async dispatch(deferred) (...) {
    %12 = mogg.async.unpack(%arg7) : ...
    %13 = mogg.rebind %12 : ... to !mogg.pointer
    ... the output, the rows dim, the context ...
    mogg.call @__mul_reduce_add_... %13, %15, ...
  }
  %11 = mgp.tensor.create %10, %9, %7
```

After **mgp-lower-primitives**: so is every primitive

```
execute (..., %arg5: !mgp.tensor<?x4096xf32, ...>) {
  %1 = mogg.routine.async (%arg5) : (%arg6: ...) {
    %12 = mogg.async.unpack(%arg6) : ...
    %13 = mgp.tensor.extract.tensor_spec %12
    %14 = mogg.async.pack(%13) : ...
  }
  ... one routine per primitive: get_dim, casts ...
  %9 = mogg.routine.async (%arg0, %8, %arg4) ... {
    %12 = mogg.async.unpack(%arg7) : index
    %13 = mogg.async.unpack(%arg8) : ...
    %14 = mogg.chain.dummy()
    %15 = mgp.buffer.alloc %14, %12, %13, align 256
    %16 = mogg.async.pack(%15) : ...
  }
  %10 = mogg.routine.async dispatch(deferred) ...
```

Profile Scope Tracing

- **profile_scope()** wraps the location of every op built inside it in a **#mogg.profile_scope**.
- Fusion keeps each op's location, so the fused kernel's location **lists both scopes**.
- **mogg-add-profile-scope-tracing** brackets each run of ops that share scopes with a **trace range** named after them.

rms_demo.py

```
with Graph.current.profile_scope("square_mean"):
  ms = F.mean(x * x, axis=-1)
with Graph.current.profile_scope("add_eps"):
  return ms + EPS
```

The ops' locations: one per scope, then fused

```
#loc9327 = #mogg.profile_scope<"square_mean",
  #loc4758>
#loc9328 = #mogg.profile_scope<"add_eps", #loc4759>
#loc9764 = loc(fused[#loc9327, #loc9328])
```

After **mogg-add-profile-scope-tracing**

```
mogg.func @__mul_reduce_add_2ba342a67ef6a164_async ... {
  ... unpack the inputs ...
  %12 = mogg.profile_scope.begin "[add_eps:square_mean] __mul_reduce_add_2ba342a67ef6a164_async"
  mogg.call @__mul_reduce_add_2ba342a67ef6a164 %6, %8, %9, %11 : ...
  ... create the output and pack it ...
  mogg.profile_scope.end %12
}
```

Error Handling

- **mogg-add-try-except-blocks** wraps a **mogg.func** body in **mogg.try** only if an op inside **can raise**.
- Kernel stubs always can: their except block re-raises with the kernel name, fusion tree and Python traces.
- In an async routine, the except block turns the error into **async errors**.

No op can raise: left as is

```
mogg.func @__primitive_0_async (...) -> index {
  %idx4 = index.constant 4
  %0 = mogg.async.unpack(%arg0) : ...
  %1 = mgp.tensor.extract.tensor_spec %0 : ...
  %2 = mgp.tensor_spec.get_dim[0] %1 : ...
  ... casts, then %5 = index.mul %4, %idx4 ...
  %6 = mogg.async.pack(%5) : index
  mogg.output %6 : !mogg.async<index>
}
```

mgp.buffer.alloc can raise: wrapped

```
mogg.func @__primitive_1_async (...) {
  %0 = mogg.try {
    %1 = mogg.async.unpack(%arg1) : index
    %2 = mogg.async.unpack(%arg2) : ...
    %3 = mogg.chain.dummy()
    %5 = mgp.buffer.alloc %3, %1, %2, align 256
    %6 = mogg.async.pack(%5) : ...
    mogg.yield %6 : ...
  } except(%err) {
    %1 = mogg.async.error %err : ...
    mogg.yield %1 : ...
  } : ...
  mogg.output %0 : ...
}
```

A Kernel Failure Points Back at the User's Python

- One message carries the **fusion tree** and both source locations: **the mean** and **the epsilon**.
- This is possible thanks to provenance information being kept throughout the compilation process and kernels wrapped in try-except clauses.

From the **try/except** blocks: the kernel, the error and its fusion tree

```
max.engine.ModelExecutionError: An error occurred in kernel named "__mul_reduce_add_2ba342a67ef6a164":
rowwise launch failed on gpu (MODULAR_DEMO_RAISE)
Fusion info:
  Rowwise
      ├── mo.mul : (...) -> ...
      ├── mo.reduce.add : (...) -> ...
      └── View ...
```

From the Python traces: each source location that fused into the kernel

```
Source Traceback:
  File ".../rms_demo.py", line 60, in main
    run = model.compile(TensorType(...))
  ...
  File ".../rms_demo.py", line 51, in forward
    ms = F.mean(x * x, axis=-1)
  ... the same path, then ...
  File ".../rms_demo.py", line 53, in forward
    return ms + EPS
```

Mojo Code Generation

- **mogg-emit-mojo** prints every stub as Mojo; **the rowwise call** becomes closures for **_rowwise**.
- **#mogg.fusion** becomes the **# FusionInfo:** comment and the **Trace** name.

`mean(x * x) + eps = sum(x * x) · 1/4096 + eps`

MOGG kernel stub

```
mogg.func @__mul_reduce_add_... (
  ..., %arg2: !mogg.dim<rows>, ...)
  attributes {..., fusionInfo =
    #mul_reduce_add_and4}
  %0 = mogg.shape.create(%arg2) ...
  mogg.with_trace {
    mogg.call.rowwise shape(%0) ... {
      mogg.rowwise.reduce @ReduceSum ...
        ...elementwise @Mul(%9, %9)
      ...elementwise @Mul(%6, %5) ...
      ...elementwise @Add(%7, %4) ...
      mogg.rowwise.emit {
        ...tensor.store %cap1[%row] = %8
```

Emitted Mojo

```
# FusionInfo:
# Rowwise
# ├── mo.mul : (...) -> ...
# ├── mo.reduce.add : (...) -> ...
# └── View ...
def __mul_reduce_add_...(..., arg__2: Int, ...) raises:
  with ...tracing.Trace[...]("Rowwise(mul, reduce.add, ..."):
    def body_____8[params : ...](row_coords: ..., ...):
      def contribute_____3[w : Int](tile: SIMD[...], ...):
        ...Mul.elementwise[dtype = DType.float32](tile, tile)
      var_____2.reduce[...ReduceSum...](contribute_____3, ...)
      def write_____5(oc: ...):
        ...Mul.elementwise[...](var_____4.acc.slice[...]...)
        ...Add.elementwise[...](var_____0, ...(9.99999997E-7))
    builtin_primitives.primitives._rowwise[...](...)
```